

Tuple Functors

Write $n: \mathbb{N}$ to mean n has type $\mathbb{N}$. (For the reader unfamiliar with type theory, it’s safe to interpret this to mean $n \in \mathbb{N}$.) For $n: \mathbb{N}$, we denote and define $\underline{n} = 0, 1, \dots, n-1$.

For $m: \mathbb{N}$, denote and define the **mtuple** functor on **Set** as follows:

- on objects
 $: \text{ if } A \text{ is a Set, then } \text{mtuple}A = (a_0, \dots, a_{m-1}) \mid a_i: A$
- on arrows
 $: \text{ if } f: A \rightarrow B \text{ is a function from the set } A \text{ to the set } B, \text{ then } \text{mtuple}f: \text{mtuple}A \rightarrow \text{mtuple}B \text{ is defined for each } (a_0, \dots, a_{m-1}) \text{ of type } \text{mtuple}A \text{ as follows:}$

$$\text{mtuple}f(a_0, \dots, a_{m-1}) = (fa_0, \dots, fa_{m-1}),$$

which inhabits the type $\text{mtuple}A$.

Notice that $\mathbf{a}$ has type $\text{mtuple}A$ iff we can represent $\mathbf{a}$ as a function of type $\underline{m} \rightarrow A$; that is, iff we can represent the **mtuple** $(a_0, \dots, a_{m-1})$ as a function, $\mathbf{a}$, where $\mathbf{a}(i) = a_i$ for each $0 \leq i < n$. Thus, we have the following equivalence of types: $\text{mtuple}A \cong \underline{m} \rightarrow A$.

Let $\mathbf{m} = (m_0, \dots, m_{n-1}): \text{ntuple}\mathbb{N}$. Define the **mtuple** functor as follows:

- on objects
 $: \text{ if } A \text{ is a Set, then}$
 $\mathbf{mtuple}A = ((a_{00}, \dots, a_{0(m_1-1)}), \dots, (a_{(n-1)0}, \dots, a_{(n-1)(m_n-1)})) \mid a_{ij}: A$
 (We may write $\mathbf{a}_i$ in place of $(a_{i0}, \dots, a_{i(m_i-1)})$, if k is clear from context.)
- on arrows
 $: \text{ if } f \text{ is a function from the set } A \text{ to the set } B, \text{ then } \mathbf{mtuple}f: \mathbf{mtuple}A \rightarrow \mathbf{mtuple}B \text{ is defined for each } (\mathbf{a}_1, \dots, \mathbf{a}_n) \text{ in } \mathbf{mtuple}A \text{ as follows:}$

$$\begin{aligned} \mathbf{mtuple}f(\mathbf{a}_1, \dots, \mathbf{a}_n) &= (\mathbf{m}_1 \text{tuple}f \mathbf{a}_1, \dots, \mathbf{m}_n \text{tuple}f \mathbf{a}_n) \\ &= ((fa_{11}, \dots, fa_{1m_1}), \dots, (fa_{n1}, \dots, fa_{nm_n})). \end{aligned}$$

Notice that $\mathbf{a}_i$ has type $\mathbf{m}_i\text{tuple}A$ iff it can be represented as a function of type $\underline{m}_i \rightarrow A$; that is, iff the tuple $(a_{i0}, \dots, a_{i(m_i-1)})$ is (the graph of) the function defined by $\mathbf{a}_i(j) = a_{ij}$ for each $0 \leq j < m_i$. Thus, if $\mathbf{m} = (m_0, \dots, m_{n-1}) : \text{ntuple}\mathbb{N}$, then $\mathbf{mtuple}A$ is the *dependent function type*,

$$\prod_{i: \underline{n}} (\underline{m}_i \rightarrow A).$$

Fork and Eval

Define $\text{fork} : (A \rightarrow B) \rightarrow (A \rightarrow C) \rightarrow A \rightarrow (B \times C)$ as follows: if $f : A \rightarrow B$, $g : A \rightarrow C$, and $a : A$, then $\text{fork}(f)(g)(a) = (f(a), g(a))$.

(A more standard definition of fork might take the domain to be $(A \rightarrow B) \times (A \rightarrow C)$, whereas we have described a “curried” version in order to support partial application.)

The fork function generalizes easily to dependent function types. Let A be a type and for each $a : A$ let B_a and C_a be types. Define the *dependent fork*, denoted by

$$\mathbf{fork} : \prod_{a:A} B_a \rightarrow \prod_{a:A} C_a \rightarrow \prod_{a:A} (B_a \times C_a),$$

as follows: if $f : \prod_{a:A} B_a$, $g : \prod_{a:A} C_a$, and $a : A$, then $\mathbf{fork}(f)(g)(a) = (f(a), g(a)) : B_a \times C_a$. Since we use a curried definition, we can partially apply $\mathbf{fork}$ and obtain the expected typing relations, viz.,

$$\mathbf{fork}(f) : \prod_{a:A} C_a \rightarrow \prod_{a:A} (B_a \times C_a) \quad \text{and} \quad \mathbf{fork}(f)(g) : \prod_{a:A} (B_a \times C_a).$$

Next, let $\mathbf{eval} : (A \rightarrow B) \times A \rightarrow B$ denote function application; that is, $\mathbf{eval}(f, a) = f(a)$, if $f : A \rightarrow B$ and $a : A$. Thus, if $h : \prod_{a:A} (C_a \rightarrow D)$, $k : \prod_{a:A} C_a$, and $a : A$, then

$$\begin{aligned} \mathbf{fork}(h)(k)(a) &= (h(a), k(a)) : (C_a \rightarrow D) \times C_a, \text{ and} \\ \mathbf{eval} \circ \mathbf{fork}(h)(k)(a) &= h(a)k(a) : D. \end{aligned}$$

General Composition of Operations on a Set

In universal algebra we deal mainly with *finitary operations on sets*. By an *n-ary operation* on the set A we mean a function $f : A^n \rightarrow A$, that takes n inhabitants of the type A and returns an element of type A .

By the equivalence of the ntuple type and the function type $\underline{n} \rightarrow A$, we may represent the type of n -ary operations on A by $(\underline{n} \rightarrow A) \rightarrow A$. Evaluating such an $f : (\underline{n} \rightarrow A) \rightarrow A$ at a tuple $a : \underline{n} \rightarrow A$ is simply function application, expressed by the usual rule (sometimes called “implication elimination” or “modus ponens”).

If we let a_i denote the value of a at i , and if we identify a with its graph (the tuple $(a_0, \dots, a_{n-1})$), then $fa = f(a_0, \dots, a_{n-1})$.

Denote and define the collection of all finitary operations on A by

$$\text{Op } A = \bigcup_{n < \omega} (A^n \rightarrow A) \cong \bigcup_{n < \omega} ((\underline{n} \rightarrow A) \rightarrow A).$$

We will now try to develop a formulation of *general function composition* that is more elegant and computationally practical than the standard formulation. Let us first briefly review the standard formulation of function composition. Let $f: (\underline{n} \rightarrow A) \rightarrow A$ be an n -ary operation on A , and suppose for each $0 \leq i < n$ we have an operation $g_i: (\underline{k_i} \rightarrow A) \rightarrow A$. Then we define $f \circ (g_0, \dots, g_{n-1})$ in the following standard way: for each

$$((a_{00}, \dots, a_{0(k_0-1)}), \dots, (a_{(n-1)0}, \dots, a_{(n-1)(k_{n-1}-1)})): A^{k_0} \times \dots \times A^{k_{n-1}},$$

$$\begin{aligned} & (f \circ (g_0, \dots, g_{n-1}))((a_{00}, \dots, a_{0(k_0-1)}), \dots, (a_{(n-1)0}, \dots, a_{(n-1)(k_{n-1}-1)})) \\ &= f(g_0(a_{00}, \dots, a_{0(k_0-1)}), \dots, g_{n-1}(a_{(n-1)0}, \dots, a_{(n-1)(k_{n-1}-1)})). \end{aligned}$$

Not only is this notation tedious, but also it lends itself poorly to computation. To improve upon it, let us first consider the tuple $(g_0, \dots, g_{n-1})$. This is an tuple of operations from $\text{Op } A$. If we denote by g the function from $\underline{n}$ to $\text{Op } A$ given by $gi = g_i$ for each $0 \leq i < n$, then g inhabits the following dependent function type:

$$\prod_{i: \underline{n}} ((\underline{k_i} \rightarrow A) \rightarrow A).$$

Next, define the function a as follows: $ai: \underline{k_i} \rightarrow A$ for each $0 \leq i < n$ and for each $j: \underline{k_i}$, $aij = a_{ij}$. Then the tuple of arguments in the expression above can be identified with the tuple $a = (a_0, \dots, a_{n-1})$ of functions. Thus a has dependent function type $\prod_{i: \underline{n}} (\underline{k_i} \rightarrow A)$, and for each $i: \underline{n}$, we have $aij = a_{ij}$.

Now, looking back at the section above, where we defined the fork and eval functions, we can see how to perform general composition using dependent types. If $g: \prod_{i: \underline{n}} ((\underline{k_i} \rightarrow A) \rightarrow A)$, and $a: \prod_{i: \underline{n}} (\underline{k_i} \rightarrow A)$, then

$$\mathbf{fork}(g)(a)(i) = (g(i), a(i)) : ((\underline{k_i} \rightarrow A) \rightarrow A) \times (\underline{k_i} \rightarrow A)$$

and $\mathbf{eval}(\mathbf{fork}(g)(a)(i)) = g(i)a(i)$ has type A . Observe that the codomain A does not depend on i , so the types $\prod_{i: \underline{n}} A$ and $\underline{n} \rightarrow A$ are equivalent. Therefore, $\mathbf{eval} \circ \mathbf{fork}(g)(a)$ has type $\underline{n} \rightarrow A$. On the other hand, we have

$$\mathbf{eval} \circ \mathbf{fork}(g) : \prod_{i: \underline{n}} (\underline{k_i} \rightarrow A) \rightarrow (\underline{n} \rightarrow A).$$

Thus, if we take an n -ary operation, $f: (\underline{n} \rightarrow A) \rightarrow A$, and an n -tuple of operations, $g: \prod_{i:\underline{n}} (\underline{k}_i \rightarrow A) \rightarrow A$, then we can *define* the **composition of f with g** as follows:

$$f[g] := f \circ (\mathbf{eval} \circ \mathbf{fork}(g)) : \prod_{i:\underline{n}} (\underline{k}_i \rightarrow A) \rightarrow A.$$

Indeed, if $a: \prod_{i:\underline{n}} (\underline{k}_i \rightarrow A)$, then $\mathbf{eval} \circ \mathbf{fork}(g)(a)$ has type $\underline{n} \rightarrow A$, which is the domain type of f ; therefore, $f(\mathbf{eval} \circ \mathbf{fork}(g)(a))$ has type A , as desired.